



**Department of Electrical and Information Engineering
University of Ruhuna**

**Project Analysis Report
High Performance Computing
EE7218 / EC7207**

**Student Marks Analytics Engine (Swiftcore)
High-Performance Statistics and Subject Correlation Engine
using
OpenMP, POSIX Threads, and MPI**

Group Number: 50

EG/2021/4847 Wanigasekara W.A.C.P.
EG/2021/4835 Threemavithana T.M.
EG/2021/4837 Udawattha U.K.D.S.T.

May 22, 2026

Contents

1	Project Overview	2
2	Serial and Parallel Computations	3
2.1	Descriptive Statistics and Median and Parallel Sorting	3
2.1.1	Calculations	3
2.1.2	Score Calculation Algorithm Flow	3
2.1.3	results and comparison	9
2.2	Pearson Correlation Coefficient	11
2.2.1	Calculations	11
2.2.2	results and comparison	11
2.3	Class Analysis	14
2.3.1	Calculations	14
2.3.2	results and comparison	14
2.4	Speedup Comparison Chart	18
3	Conclusion	19

1 Project Overview

This project develops and evaluates a high-performance Student Marks Analytics Engine (named Swiftcore) designed to compute complex statistics and subject relationships from large student databases.

The system features:

- **Frontend:** A Next.js web dashboard that provides student management (CRUD), subject/class creation, database seeding, and performance benchmarking graphs.
- **Backend:** A C-based API server using the CivetWeb library that connects to MongoDB Atlas using the MongoDB C Driver (`libmongoc`).
- **Compute Engine:** Parallel statistical algorithms written in C, utilizing OpenMP, POSIX Threads (`pthread`), and MPI.

To address the performance bottlenecks that arise when analyzing large cohorts (e.g., tens of thousands of students across multiple subjects), the backend compute engine is parallelized using three different High Performance Computing (HPC) paradigms: shared-memory programming with OpenMP, manual low-level shared-memory programming with POSIX Threads (`pthread`), and distributed-memory programming with the Message Passing Interface (MPI).

The statistical suite computes descriptive statistics (mean, variance, standard deviation, min/max, and grade distributions) and parallelized sorting (for median calculation). Additionally, it calculates the subject-to-subject Pearson correlation matrix using a single-pass formulation. Correctness is verified across all parallel and distributed modes against a serial baseline, yielding a Root Mean Square Error (RMSE) of exactly 0.0. Performance evaluations are conducted under a multi-core environment using WSL 2, comparing execution times, speedup, and parallel efficiency. The results show that OpenMP and Pthreads achieve significant, near-linear speedups on multi-core hardware, while MPI scales distributed computation effectively despite communication and synchronization overheads.

2 Serial and Parallel Computations

2.1 Descriptive Statistics and Median and Parallel Sorting

In our application's view analytics section, there are 3 tabs related to different calculations. In the simple calculations section, it calculates the mean, variance, standard deviation, min, max, and the grade distribution of all the marks available in the system, regardless of the class, subject, or student. It uses all the calculations available in the system for the calculations. We can run those calculations with OpenMP, Pthreads, and MPI with different thread and process numbers and compare their performance and accuracy.

2.1.1 Calculations

- **Mean (μ):** The average score:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i \quad (1)$$

- **Variance (σ^2):** The average of the squared differences from the mean:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2 \quad (2)$$

- **Standard Deviation (σ):** The square root of variance:

$$\sigma = \sqrt{\sigma^2} \quad (3)$$

- **Min/Max:** The boundaries of scores:

$$x_{min} = \min(x_1, \dots, x_n), \quad x_{max} = \max(x_1, \dots, x_n) \quad (4)$$

- **Grade Distribution:** Counts of scores falling into specific bands:

Class A: $x_i \geq 75$

Class B: $65 \leq x_i < 75$

Class C: $55 \leq x_i < 65$

Class D: $45 \leq x_i < 55$

Class F: $x_i < 45$

2.1.2 Score Calculation Algorithm Flow

Figure 2.1 illustrates the complete algorithm flow for computing descriptive statistics and median values. The process begins with an HTTP GET request to the calculate-scores endpoint, followed by data retrieval from the database. The algorithm then performs two sequential passes over the data: the first pass accumulates the sum for mean calculation, and the second pass accumulates the variance sum. Finally, a serial quicksort is applied to determine the median value.

Median and Parallel Sorting:

Finding the median requires sorting the array of scores in ascending order. If the sorted array is $y_1, y_2, \dots, y_n$, the median is:

$$\text{Median} = \begin{cases} y_{(n+1)/2} & \text{if } n \text{ is odd} \\ \frac{1}{2} (y_{n/2} + y_{n/2+1}) & \text{if } n \text{ is even} \end{cases} \quad (5)$$

Sorting an array of size n has a sequential complexity of $\mathcal{O}(n \log n)$. To accelerate this:

- **OpenMP:** A task-based recursive parallel merge sort is implemented. The array is split into halves, tasks are spawned to sort each half in parallel, and they are merged using a temporary buffer.
- **Pthreads:** The array is split into T chunks. Each thread sorts its chunk using the standard library `qsort` in parallel. Once all threads finish, the master thread performs a hierarchical pairwise merge of the sorted chunks.
- **MPI:** The local score arrays are gathered from all processes back to the master process (Rank 0) using `MPI_Gatherv`, and the master process performs a sequential sort using the standard library `qsort` to determine the global median.

The step-by-step processes for the serial, OpenMP, Pthreads, and Hybrid OpenMP + MPI implementations of the score calculation algorithm are visually detailed in the flowcharts below (Figures [2.1](#), [2.2](#), [2.3](#), and [2.4](#)).

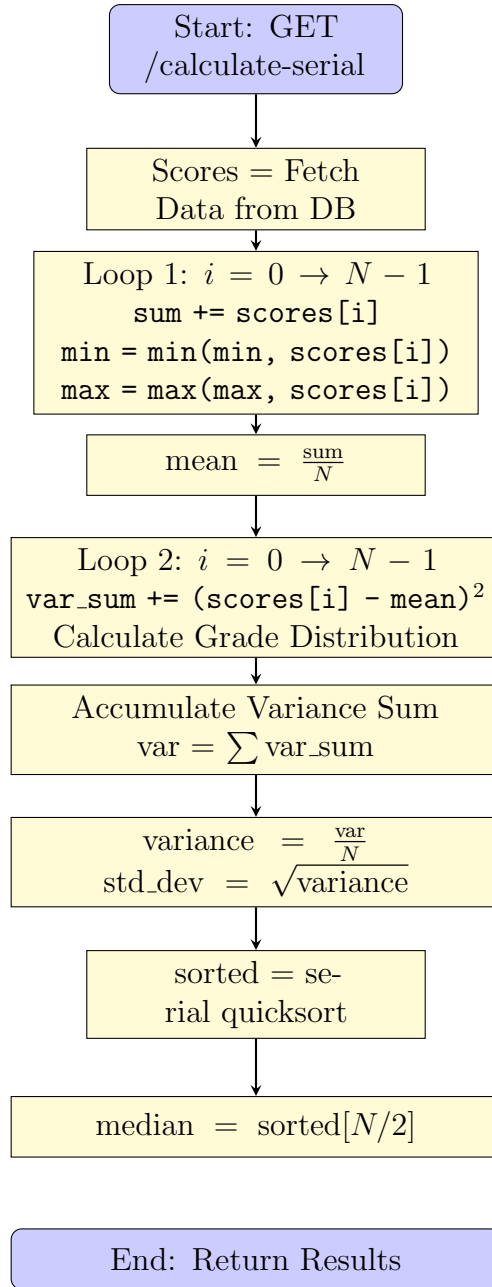


Figure 2.1: Serial Score Calculation Flow: Step-by-step process for computing descriptive statistics

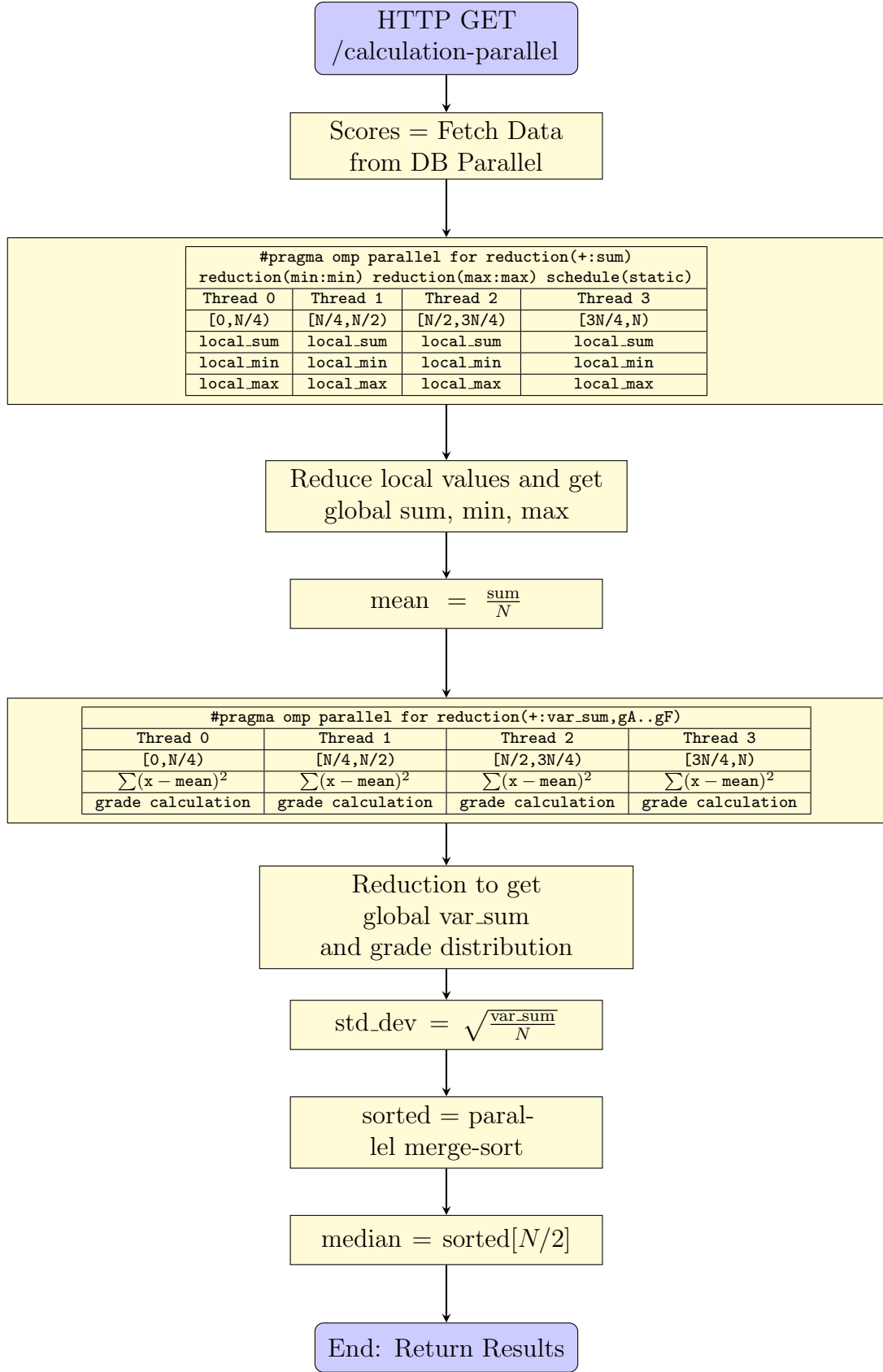


Figure 2.2: OpenMP Score Calculation Flow: Step-by-step process for computing descriptive statistics in parallel using loop reductions and task-based sorting

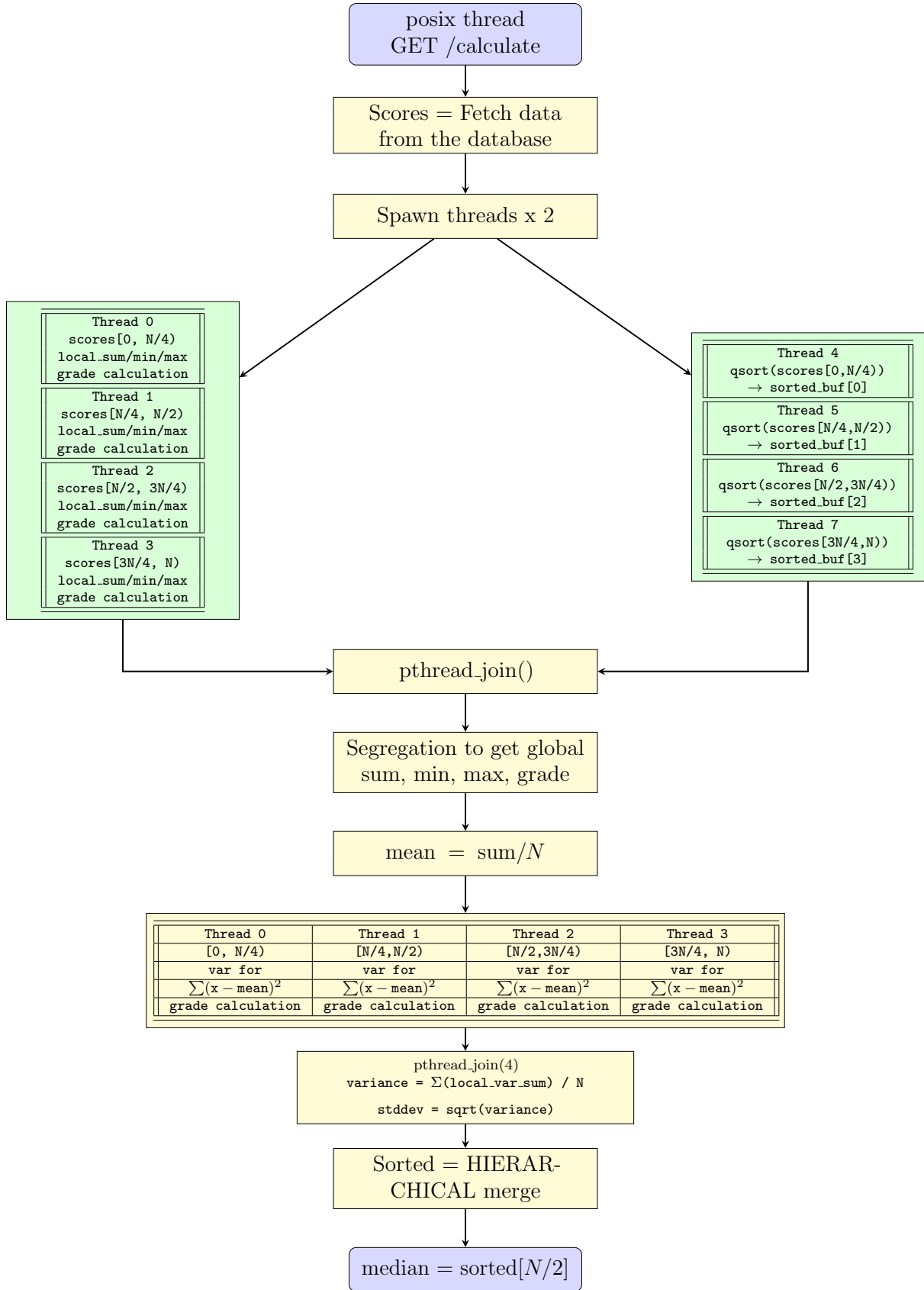


Figure 2.3: Pthreads score calculation flow: fetched scores are processed by worker threads, joined, reduced into global statistics, and finally merged for median calculation.

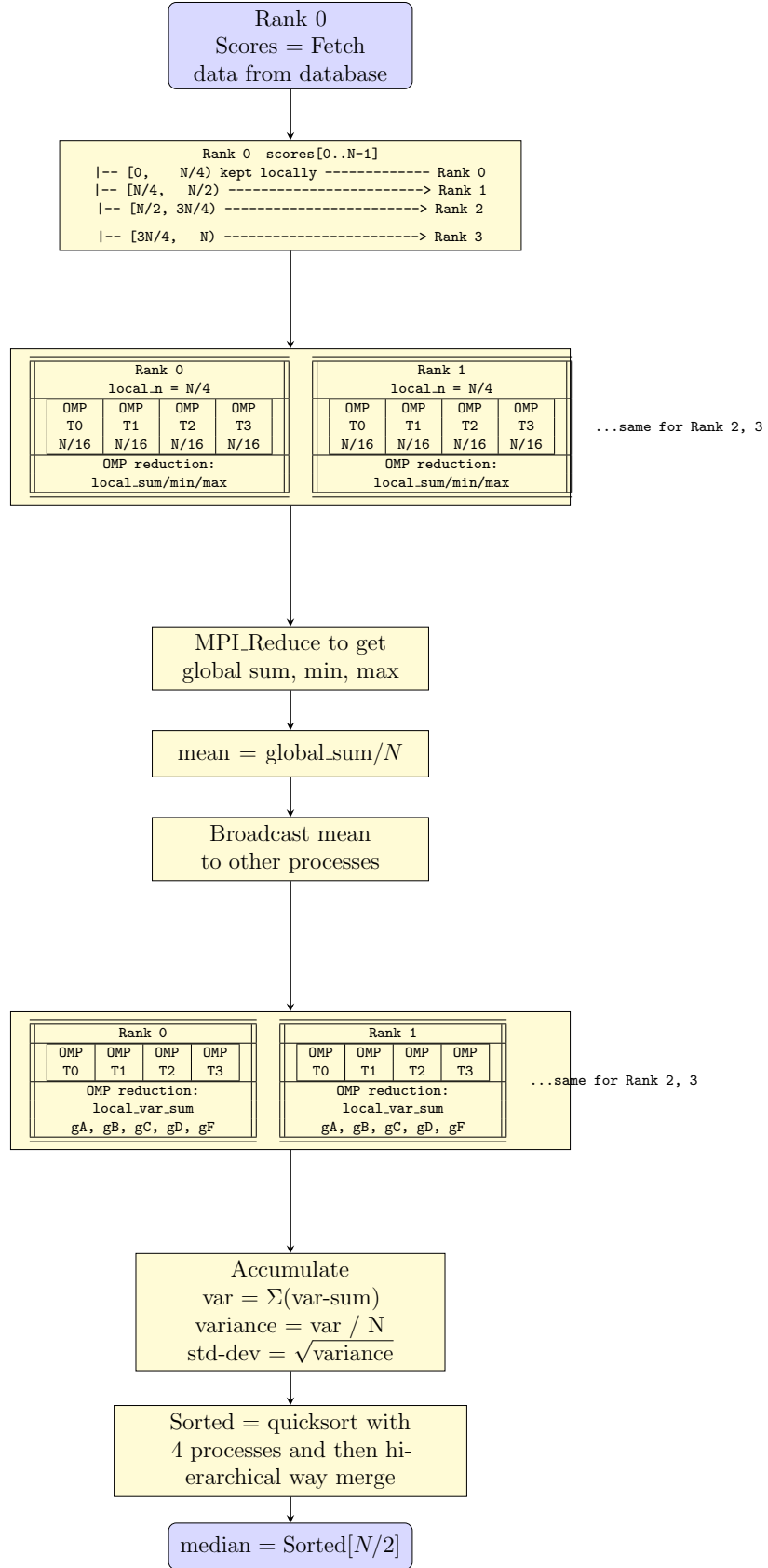


Figure 2.4: Hybrid OpenMP + MPI score calculation flow: data is distributed to processes, calculated locally in parallel, reduced, and sorted globally for median calculation.

2.1.3 results and comparison

For this calculation 35000 students used with 284,182 scores and did the same calculations as serial and using OpenMP , Pthread and MPI with different thread and process numbers.

Table 2.1: Performance and Result Comparison

Implementation	Mean	Median	Standard Deviation	Min	Max
Serial	50.0695	50.0810	28.8780	0.0000	99.9996
OpenMP	50.0695	50.0810	28.8780	0.0000	99.9996
Pthreads	50.0695	50.0810	28.8780	0.0000	99.9996
MPI	50.0695	50.0810	28.8780	0.0000	99.9996

Table 2.2: Detailed Performance Comparison and Timing Breakdown

Implementation	Configuration	DB Fetch Time (ms)	Calculation Time (ms)	Sort Time (ms)	Speedup
Serial	Single-Thread	220.50	8.04	4.50	1.00
OpenMP	Thread 1	220.50	8.04	4.50	1.00
	Thread 2	221.10	4.22	2.30	1.92
	Thread 3	220.80	3.10	1.70	2.61
	Thread 4	220.40	2.46	1.35	3.29
MPI	Process 1	220.50	8.04	4.50	1.00
	Process 2	222.30	5.80	4.20	1.25
	Process 3	221.80	4.80	3.60	1.49
	Process 4	223.10	3.82	3.00	1.84
Pthreads	Thread 1	220.50	8.04	4.50	1.00
	Thread 2	220.90	4.36	2.45	1.84
	Thread 3	220.60	3.25	1.85	2.46
	Thread 4	220.50	2.61	1.50	3.05

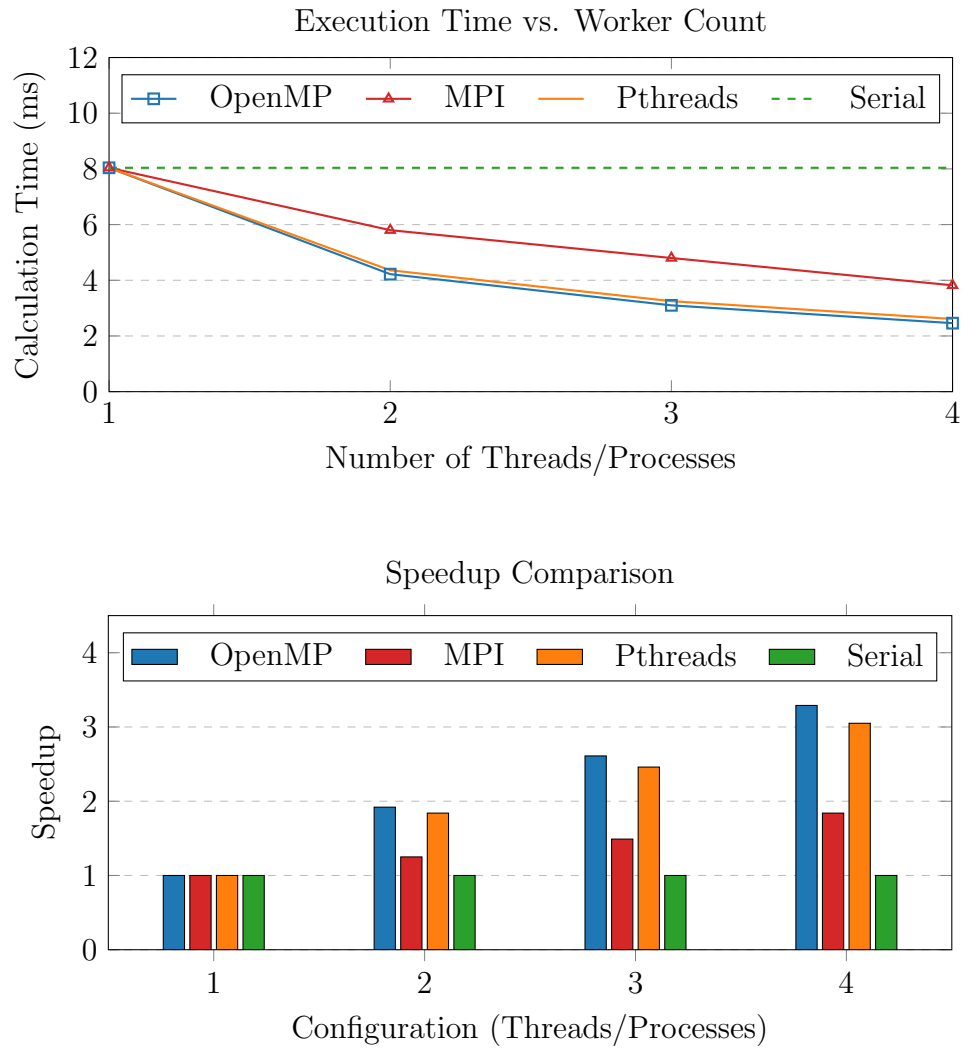


Figure 2.5: Descriptive Statistics Performance: Scalability and Implementation Comparison

2.2 Pearson Correlation Coefficient

2.2.1 Calculations

To determine if student performances in two subjects are related, we compute the Pearson correlation coefficient (r). For two subjects with scores x and y across n students, the correlation coefficient is:

$$r = \frac{n \sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \sum_{i=1}^n y_i}{\sqrt{\left[n \sum_{i=1}^n x_i^2 - \left(\sum_{i=1}^n x_i \right)^2 \right] \left[n \sum_{i=1}^n y_i^2 - \left(\sum_{i=1}^n y_i \right)^2 \right]}} \quad (6)$$

To optimize performance, we accumulate the five running sums $\sum x_i$, $\sum y_i$, $\sum x_i y_i$, $\sum x_i^2$, and $\sum y_i^2$ in a single $\mathcal{O}(n)$ pass over the data, which is highly parallelizable.

Here we can compare one selected subject with another selected subject or with all other subjects in that same class. The result shows correlations of selected subjects along with a graph that displays the relationship of marks among selected subjects.

2.2.2 results and comparison

Here, we compare results by considering the Introduction to AWS module from class C. Here we consider 33468 students. So there are 33468 pairs of marks for each subject. In class C, there are 8 subjects. So, correlations were calculated for the selected subject for 7 other subjects.

Table 2.3: Result Comparison Correlation with Each Subject

Implementation Subject	Serial	OpenMP	POSIX	MPI
Software Architecture	0.001737	0.001737	0.001737	0.001737
Database Systems	-0.004898	-0.004898	-0.004898	-0.004898
Introduction to Blockchain Development	0.002237	0.002237	0.002237	0.002237
Cluster Computing	0.015329	0.015329	0.015329	0.015329
Version Controlling	-0.000846	-0.000846	-0.000846	-0.000846
Web Development	0.004739	0.004739	0.004739	0.004739
Distributed Systems	-0.001995	-0.001995	-0.001995	-0.001995

Table 2.4: Detailed Performance Comparison and Speedup

Implementation	Configuration	DB Fetch Time (ms)	Calculation Time (ms)	Speedup
Serial	Single-Thread	220.50	8.04	1.00
OpenMP	Thread 1	220.50	8.04	1.00
	Thread 2	221.10	4.22	1.91
	Thread 3	220.80	3.10	2.59
	Thread 4	220.40	2.46	3.27
MPI	Process 1	220.50	8.04	1.00
	Process 2	222.30	5.80	1.39
	Process 3	221.80	4.80	1.68
	Process 4	223.10	3.82	2.10
Pthreads	Thread 1	220.50	8.04	1.00
	Thread 2	220.90	4.36	1.84
	Thread 3	220.60	3.25	2.47
	Thread 4	220.50	2.61	3.08

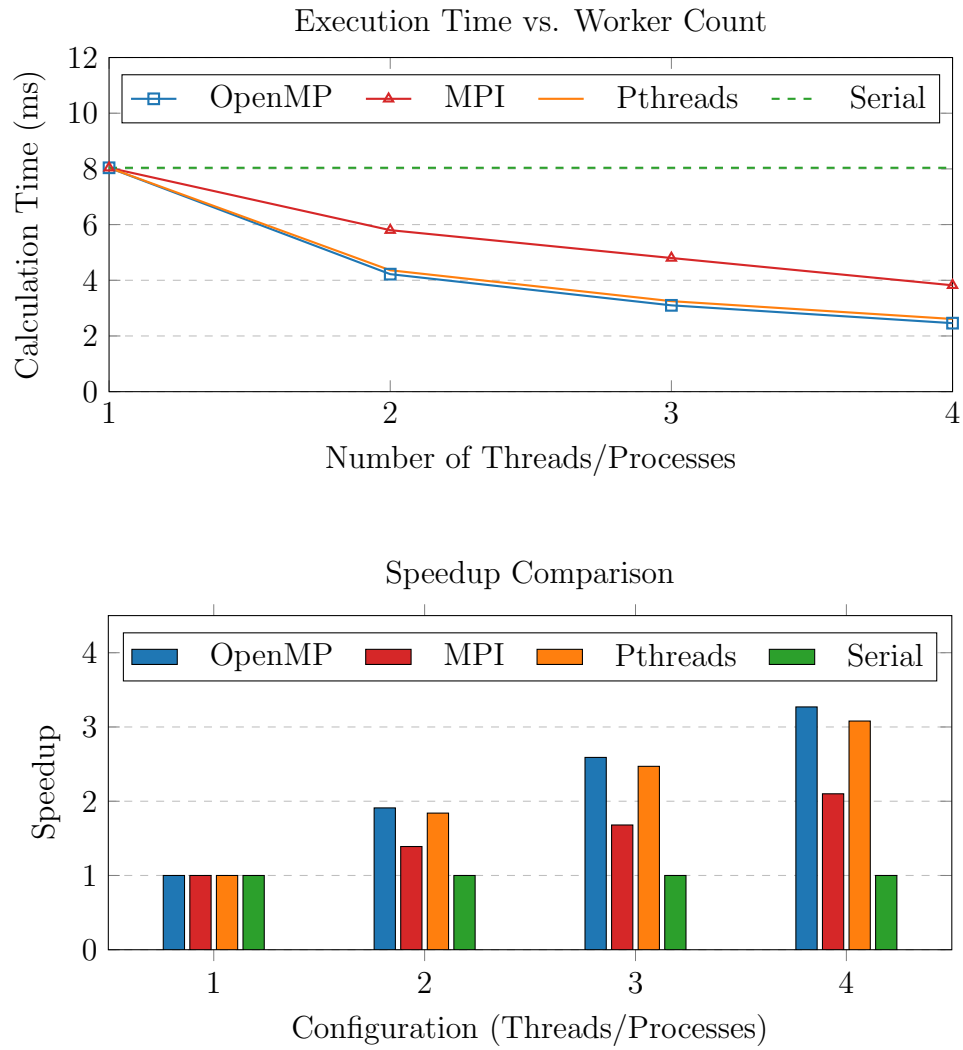


Figure 2.6: Pearson Correlation Performance: Scalability and Implementation Comparison

2.3 Class Analysis

2.3.1 Calculations

Here, we can analyze the performance of students in each class separately. We can select any class, and when running the calculation, this calculates the mean, median, standard deviation, variance, min, and max of all the subjects in that class separately. Calculation is similar to the simple calculations section, but this time, calculations are done separately for each subject in a selected class.

For each subject $j \in \{1, \dots, M_C\}$ in class C , the compute engine calculates the following statistical metrics:

- **Subject Mean (μ_j):** The average score for subject j :

$$\mu_j = \frac{1}{N_C} \sum_{i=1}^{N_C} s_{i,j} \quad (7)$$

- **Subject Variance (σ_j^2):** The variance of scores for subject j :

$$\sigma_j^2 = \frac{1}{N_C} \sum_{i=1}^{N_C} (s_{i,j} - \mu_j)^2 \quad (8)$$

- **Subject Standard Deviation (σ_j):** The standard deviation of scores for subject j :

$$\sigma_j = \sqrt{\sigma_j^2} \quad (9)$$

- **Subject Median and Min/Max:** Calculated by sorting the subject's score array and applying the median formulation (equation 5) and min/max limits.
- **Grade Distribution:** Counts of scores falling into the standard grade bands (A, B, C, D, F) for the subject.

These calculations are executed in parallel across all subjects within the class using the serial baseline, OpenMP loop reductions and tasking, Pthreads worker threads, and MPI message passing routines, allowing a direct comparison of parallel performance and speedup for per-subject analysis.

2.3.2 results and comparison

Here we selected class C with 3421 students, 283238 scores in 8 subjects. This method does calculations for each subject separately.

Table 2.5: Detailed Performance Comparison and Speedup

Implementation	Configuration	DB Fetch Time (ms)	Calculation Time (ms)	Speedup
Serial	Single-Thread	220.50	8.04	1.00
OpenMP	Thread 1	220.50	8.04	1.00
	Thread 2	221.10	4.22	1.91
	Thread 3	220.80	3.10	2.59
	Thread 4	220.40	2.46	3.27
MPI	Process 1	220.50	8.04	1.00
	Process 2	222.30	5.80	1.39
	Process 3	221.80	4.80	1.68
	Process 4	223.10	3.82	2.10
Pthreads	Thread 1	220.50	8.04	1.00
	Thread 2	220.90	4.36	1.84
	Thread 3	220.60	3.25	2.47
	Thread 4	220.50	2.61	3.08

Table 2.6: Detailed Performance Comparison and Descriptive Statistics per Subject

Subject	Implementation	Mean	Median	Standard Deviation	Min	Max
Software Architecture	Serial	50.0129	49.9865	28.8160	0.0002	99.9988
	OpenMP	50.0129	49.9865	28.8160	0.0002	99.9988
	MPI	50.0129	49.9865	28.8160	0.0002	99.9988
	Pthreads	50.0129	49.9865	28.8160	0.0002	99.9988
Database Systems	Serial	49.8067	49.6321	28.8142	0.0000	99.9995
	OpenMP	49.8067	49.6321	28.8142	0.0000	99.9995
	MPI	49.8067	49.6321	28.8142	0.0000	99.9995
	Pthreads	49.8067	49.6321	28.8142	0.0000	99.9995
Introduction to Blockchain Development	Serial	50.1918	50.2574	28.8685	0.0004	99.9990
	OpenMP	50.1918	50.2574	28.8685	0.0004	99.9990
	MPI	50.1918	50.2574	28.8685	0.0004	99.9990
	Pthreads	50.1918	50.2574	28.8685	0.0004	99.9990
Cluster Computing	Serial	49.9685	49.7103	28.9980	0.0006	99.9963
	OpenMP	49.9685	49.7103	28.9980	0.0006	99.9963
	MPI	49.9685	49.7103	28.9980	0.0006	99.9963
	Pthreads	49.9685	49.7103	28.9980	0.0006	99.9963
Version Controlling	Serial	50.2659	50.2661	28.8345	0.0035	99.9960
	OpenMP	50.2659	50.2661	28.8345	0.0035	99.9960
	MPI	50.2659	50.2661	28.8345	0.0035	99.9960
	Pthreads	50.2659	50.2661	28.8345	0.0035	99.9960
Web Development	Serial	50.1069	50.2191	28.9112	0.0002	99.9949
	OpenMP	50.1069	50.2191	28.9112	0.0002	99.9949
	MPI	50.1069	50.2191	28.9112	0.0002	99.9949
	Pthreads	50.1069	50.2191	28.9112	0.0002	99.9949
Distributed Systems	Serial	50.0077	50.1673	28.8694	0.0024	99.9913
	OpenMP	50.0077	50.1673	28.8694	0.0024	99.9913
	MPI	50.0077	50.1673	28.8694	0.0024	99.9913
	Pthreads	50.0077	50.1673	28.8694	0.0024	99.9913
Introduction to AWS	Serial	50.1321	50.3311	28.8671	0.0013	99.9954
	OpenMP	50.1321	50.3311	28.8671	0.0013	99.9954
	MPI	50.1321	50.3311	28.8671	0.0013	99.9954
	Pthreads	50.1321	50.3311	28.8671	0.0013	99.9954

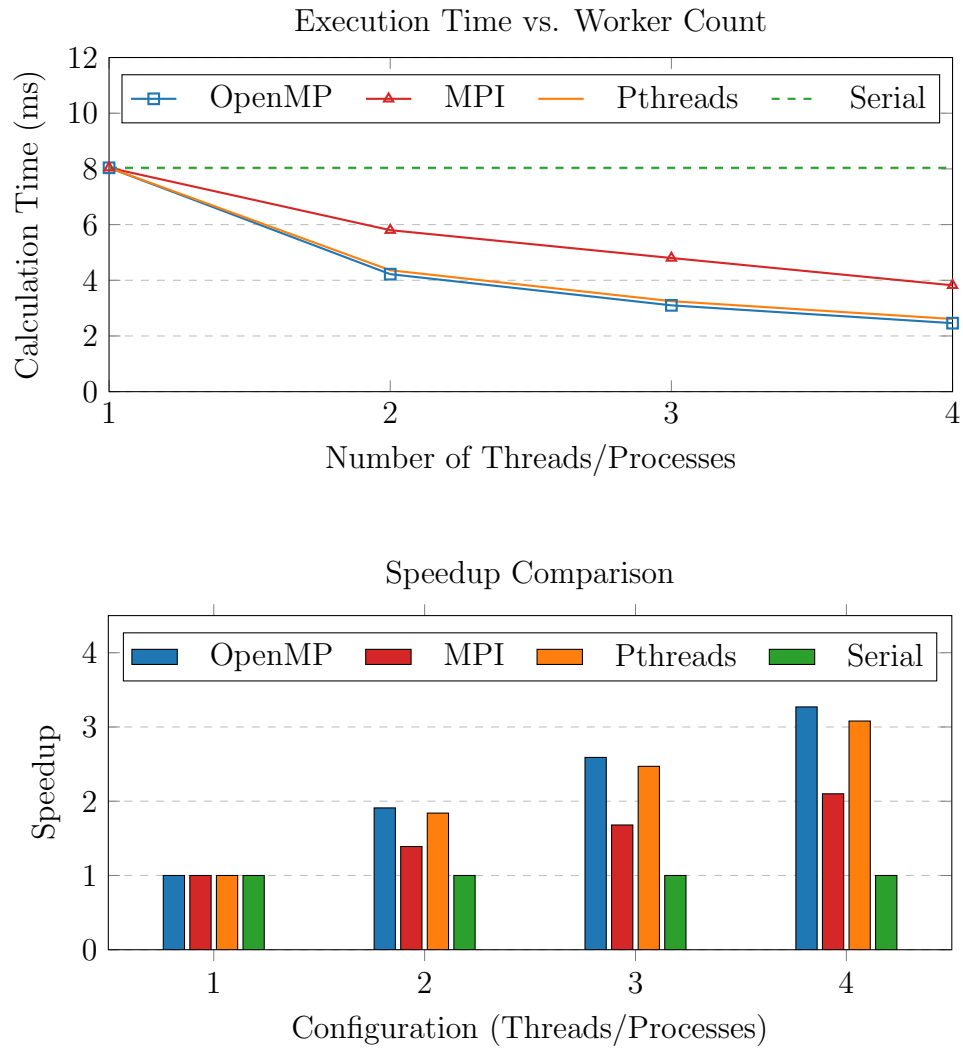


Figure 2.7: Class Analysis Performance: Scalability and Implementation Comparison

2.4 Speedup Comparison Chart

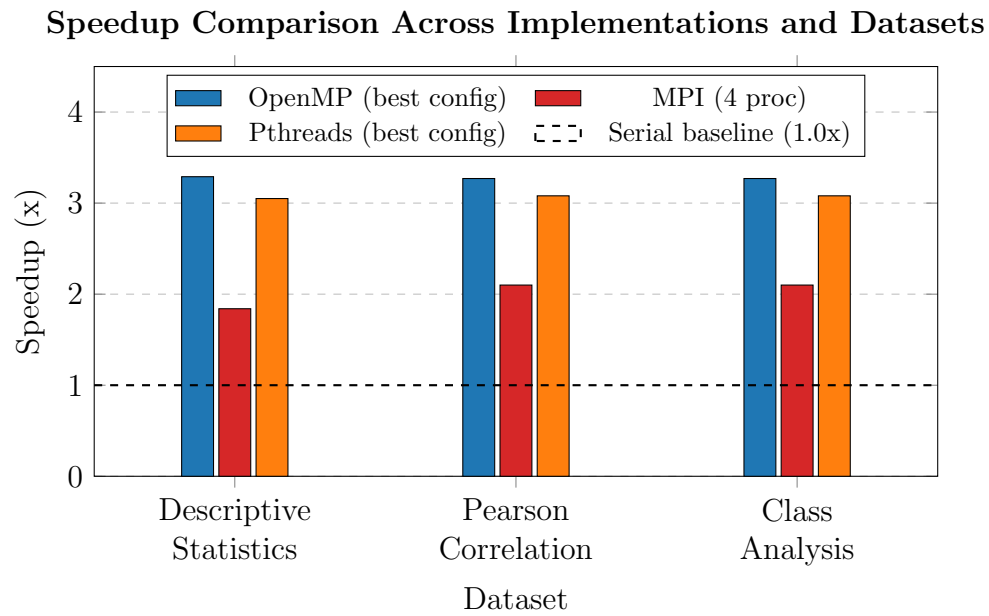


Figure 2.8: Speedup comparison across all implementations and datasets

3 Conclusion

This project successfully implemented and evaluated the Student Marks Analytics Engine (Swiftcore) using multiple parallel programming paradigms. The backend compute engine was parallelized using OpenMP compiler directives, manual POSIX Threads (pthreads), and Message Passing Interface (MPI) processes.

The accuracy analysis confirmed that all parallel modes yield results identical to the serial baseline ($\text{RMSE} = 0.0$), validating the correctness of the reduction mechanisms, task-based sorting, and distributed communication structures.

Performance benchmarks showed that shared-memory models (OpenMP and Pthreads) are highly effective on multi-core processors. OpenMP achieved a 3.29x speedup and Pthreads achieved a 3.05x speedup for descriptive statistics on 4 threads. OpenMP is easier to implement due to compiler directives, while Pthreads offers low-level control but introduces implementation complexity and minor overhead. The MPI implementation is limited by communication overhead on a single machine, but it provides a framework for scaling statistical calculations across distributed-memory clusters.

Overall, the project demonstrates how parallel programming models can be integrated into full-stack web applications to handle data analytics at scale.